

Business Intelligence - LAB 3

Juan Pablo Castro - 202222086

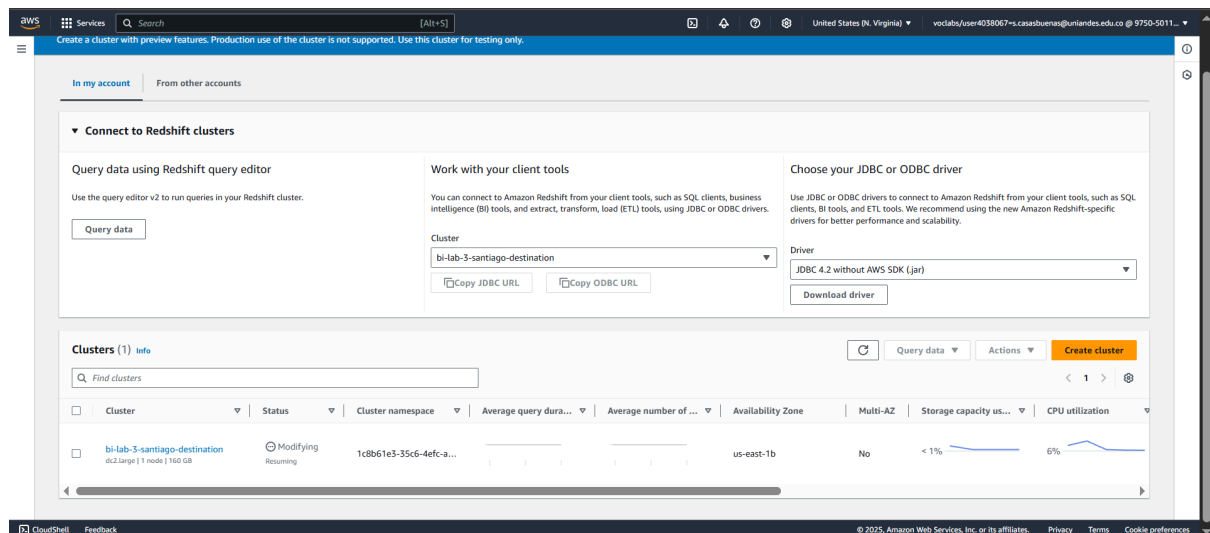
Santiago Casasbuenas - 202214932

Manuel Gómez - 202020415

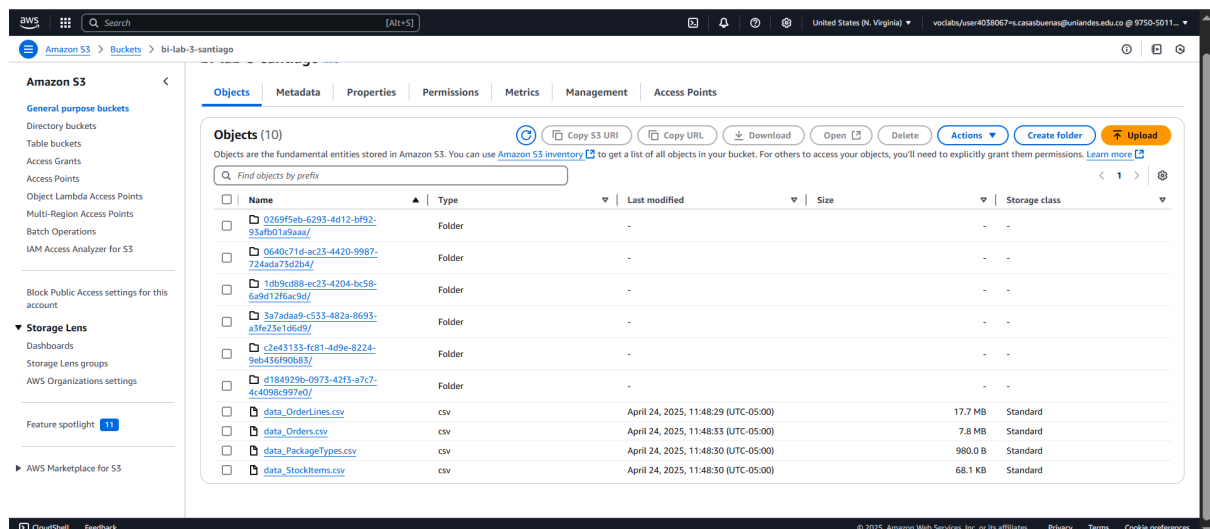
Etapas 1. Activación de Servicios

Configuración y despliegue del ETL

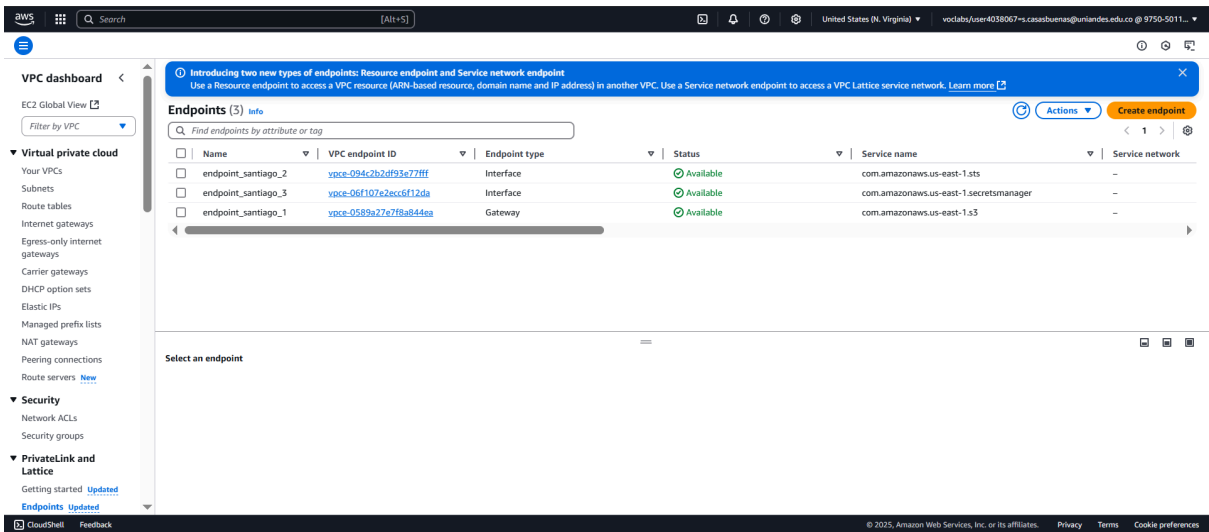
Evidencia Creación de Cluster



Evidencia de Buckets con Archivos



Evidencia VPC Endpoints



Evidencia ETL Inicial

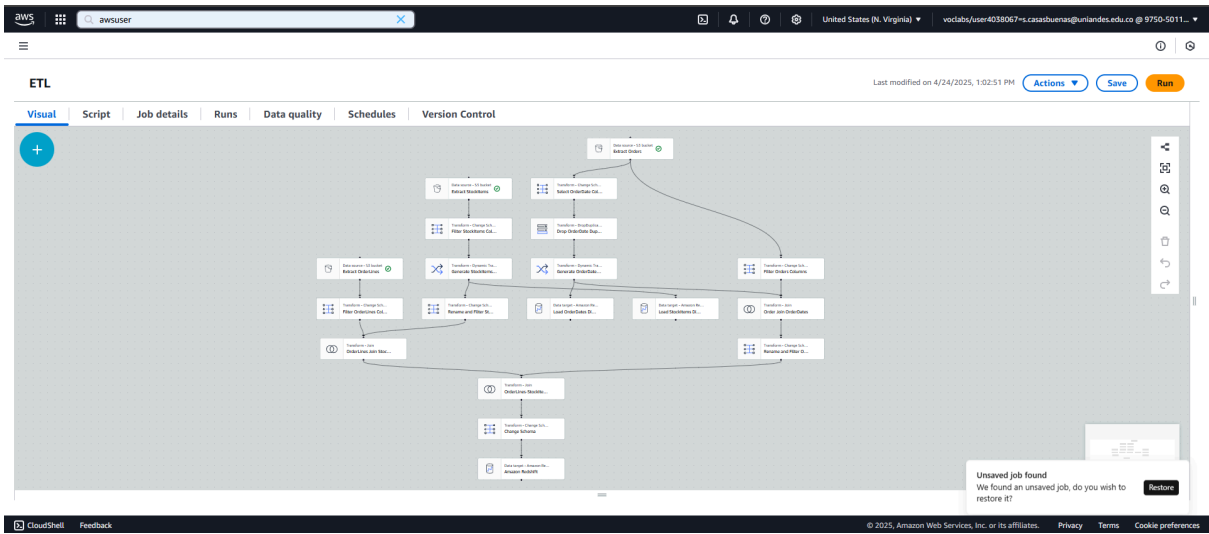
Archivo json con datos remplazados

Código ID: 975050117748

TempDir: //bi-lab-3-santiago

Link a archivo JSON: [link_etl_inicial](#)

Visual ETL inicial



Evidencia de ejecución de ETL

The screenshot displays the AWS Glue console interface. At the top, the 'ETL' section is active, showing a table of job runs. The table has columns for Run status, Retries, Start time (Local), End time (Local), Duration, Capacity (DPU), Worker type, and Glue version. The first run is 'Succeeded' with 0 retries, starting at 04/30/2025 09:56:57 and ending at 04/30/2025 10:00:14, with a duration of 2 m 56 s. Below the table, the 'Run details' section is expanded, showing job name 'ETL', job ID 'j-5184843b4970ceb1bea5a02c20d24726de47e6b9ef356d1e0f6f00', and various metrics like start-up time (21 seconds), execution time (2 minutes 56 seconds), and security configuration. The console also shows input arguments, logs, and run insights.

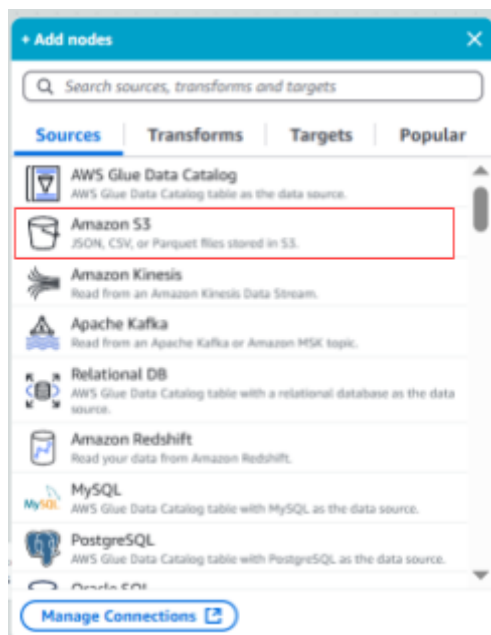
Etapas 2. Proceso ETL

Análisis de Subprocesos

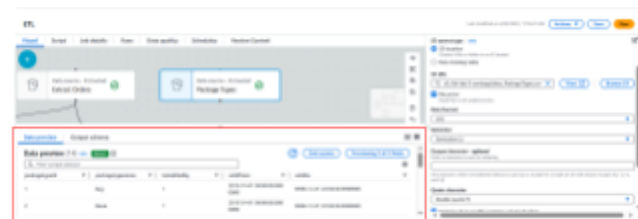
2.1 Analice los diferentes subprocessos del ETL proporcionado. Identifique los nodos o bloques para la extracción de datos desde S3, los diferentes filtros, *joins*, agregaciones y generadores de *PKs* que se encuentran implementados, así como los requeridos para la carga de datos a Redshift. Describa en sus propias palabras el proceso general implementado.

Bloques para la extracción de datos desde S3

Podemos identificar tres nodos de extracción para cada uno de los archivos que tenemos cargados (proximamente 4 cuando carguemos el ultimo). Estos nodos se encuentran definidos en AWS como “Amazon S3” y funcionan como buckets en los que está contenida la información (son la representación de como se verán las tablas). Su configuración requiere de un URL, el cual hace referencia al archivo csv al cual este debe acceder dentro de AWS. Se ve de la siguiente manera:

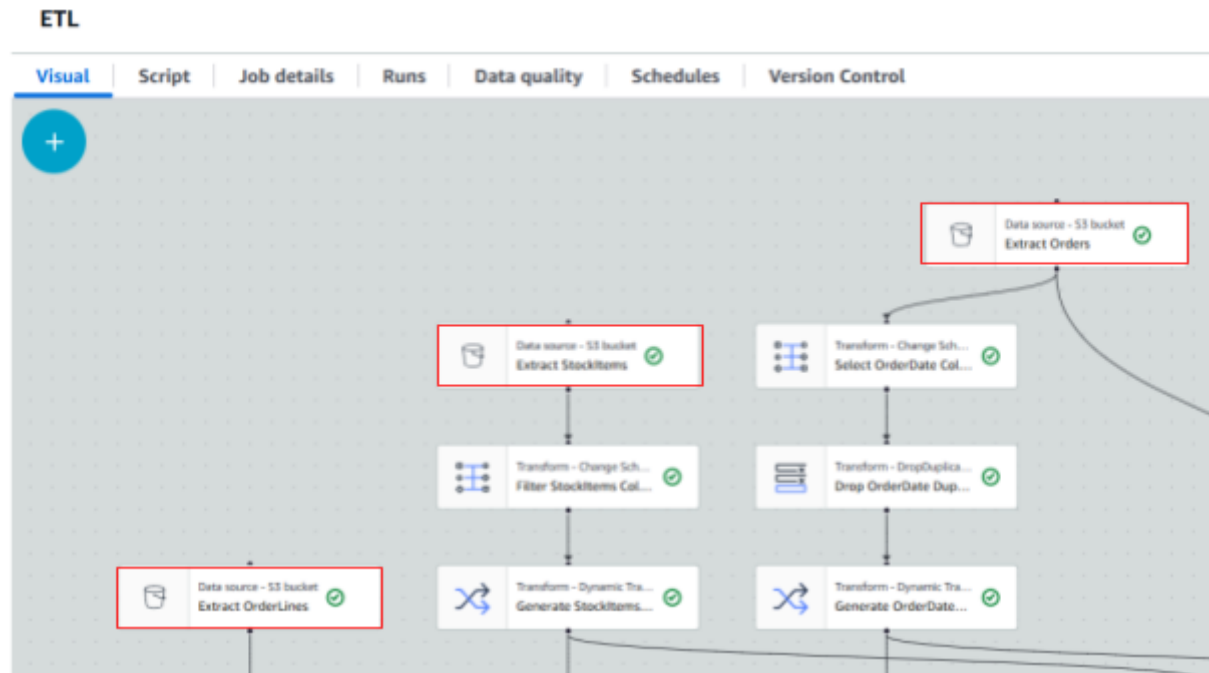


Al seleccionar la opción, obtendremos acceso a la configuración de como queremos que sea extraída la información desde el archivo csv. Aquí podremos cambiar los parámetros como el formato de la información (CSV), delimitador (semicolon, coma y demás).



Aquí se puede apreciar que, si la conexión se establece de forma apropiada, podremos ver un “preview” de como se ven los datos en formato de tabla.

En cuanto a los datos del ETL inicial, sabemos que estos nodos son los responsables de extraer los datos de los archivos “data_OrderLines.csv”, “data_StockItems.csv” y “data_StractOrders.csv”. Cad uno está configurado para obtener la información de los archivos y posteriormente subirlos a redshift.



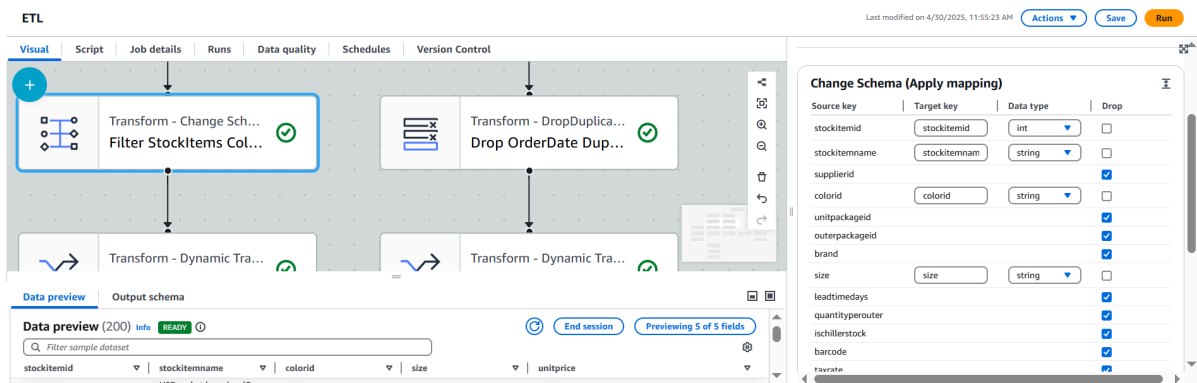
Filtros para las tablas o “change schema node”

Los nodos de filtros, específicamente el de transformación o “change schema”, como su nombre lo indica, se encarga de mapear en un formato específico los datos de la tabla que fue previamente definida por el nodo Amazon S3. Por ejemplo, en el caso de las fechas, puede que necesitemos utilizar un formato específico para poder organizarlas. En el caso de “Orders” por ejemplo, se usa un nodo change schema para aislar las fechas y poder hacer operaciones sobre estas más adelante.

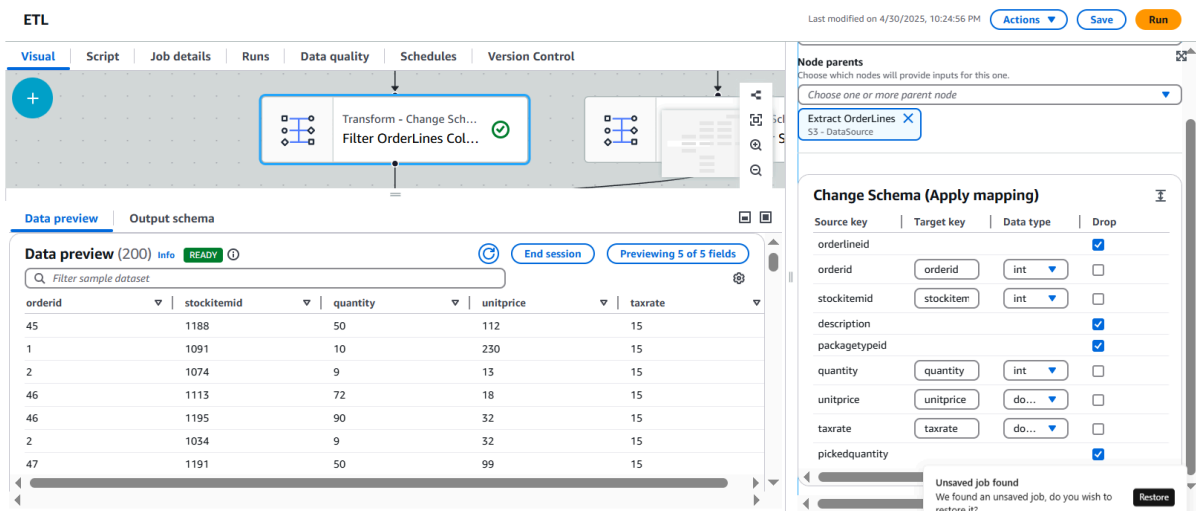
ETL

Source key	Target key	Data type	Drop
orderid	orderid	date	☒
customerid	customerid	drop	☒
salespersonpersonid	salespersonpersonid	drop	☒
pickedbypersonid	pickedbypersonid	drop	☒
contactpersonid	contactpersonid	drop	☒
backorderorderid	backorderorderid	drop	☒
orderdate	orderdate	date	☐
expecteddeliverydate	expecteddeliverydate	drop	☒
customerpurchaseord	customerpurchaseord	drop	☒
isundersupplybackord	isundersupplybackord	drop	☒
comments	comments	drop	☒
deliveryinstructions	deliveryinstructions	drop	☒
internalcomments	internalcomments	drop	☒
pickingcompletedwh	pickingcompletedwh	drop	☒
lasteditedby	lasteditedby	drop	☒

En el caso de StockItems, se hace un mapeo sobre las columnas que tienen que ver con los objetos del inventario como el ID de ese ítem, el nombre, el color y el tamaño. Esto, como se mencionó anteriormente, se hace para poder realizar operaciones sobre estos elementos más adelante.



Por último, en el caso de OrderLines, sucede algo similar. Se escogen todos los parámetros que son necesarios para las líneas de una orden.



Nodos join y generación de llaves primarias

Estos nodos sirven para crear relaciones entre los filtros que se construyeron anteriormente. En este caso, tenemos dos joins principales que juntan las transformaciones iniciales. Luego, hay un join que junta todas las transformaciones, este es de suma importancia porque nos garantiza que tendremos toda la información en un formato que nos permita hacer consultas sobre los datos que fueron transformados con un fin específico.

Se pueden hacer diferentes tipos de joins, aunque para el caso de este ETL, solo se usan inner joins, lo que significa que se están seleccionando todas las filas de ambos datasets que cumplan la condición del join. Es decir, para el caso de "Order Join OrderDates" se tendrán en cuenta para este join solamente las tablas que pertenezcan al filtro/transformación de "Filter Order Columns", y las llaves primarias creadas para OrderDates.

The screenshot displays the ETL tool's 'Transform' node configuration for 'Join Order Dates'. The 'Data preview' window shows a table with the following data:

orderdate_1	OrderDateKey	orderid	orderdate
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	1	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	2	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	3	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	4	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	5	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	6	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	7	2015-01-01
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	8	2015-01-01

The 'Transform' panel shows the following configuration:

- Name:** Order Join OrderDates
- Join type:** Inner join
- Join conditions:** orderdate_1 = orderdate

Si nos fijamos en la imagen, vemos que el nodo de join actual cuenta con dos conexiones entrante de dos nodos. El primero es el filtro de transformación a partir de mapeo (Filter Orders Columns) y el segundo es la transformación dinámica UUID (Generate OrderDates Dimensions PK). Nos interesa hablar más a fondo de este último nodo, ya que es el responsable de añadir una columna UUID (Universally Unique Identified) la cual asigna un identificador único a cada fila de la tabla a la que se le aplica la transformación. Este UUID resulta siendo la llave primaria de la tabla en sí. Para confirmar esto, si nos metemos al nodo Generate OrderDates Dimension PK, nos daremos cuenta de la información de llaves primarias generadas para las diferentes fechas que existen dentro de las fechas de envío:

The screenshot displays the ETL tool's 'Transform' node configuration for 'Generate OrderDates Dimension PK'. The 'Data preview' window shows a table with the following data:

orderdate	OrderDateKey	OrderDate
2015-01-01	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	2015-01-01
2015-01-02	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	2015-01-02
2015-01-03	a3f8b4d0-c9b3-4aaf-085-7-5d7f16b4d0-514	2015-01-03

The 'Transform' panel shows the following configuration:

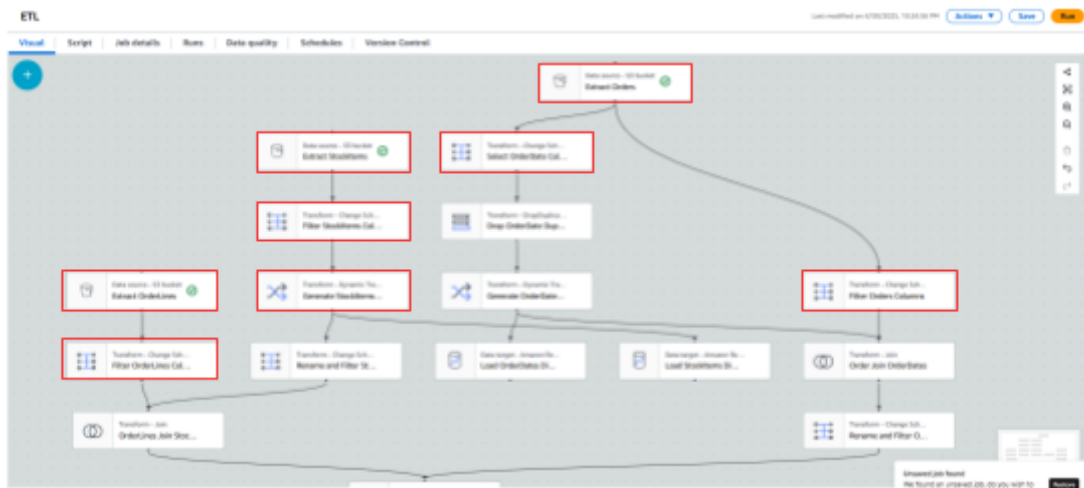
- Name:** Generate OrderDates Dimension PK
- UUID Column name:** OrderDateKey

Descripción general del proceso implementado

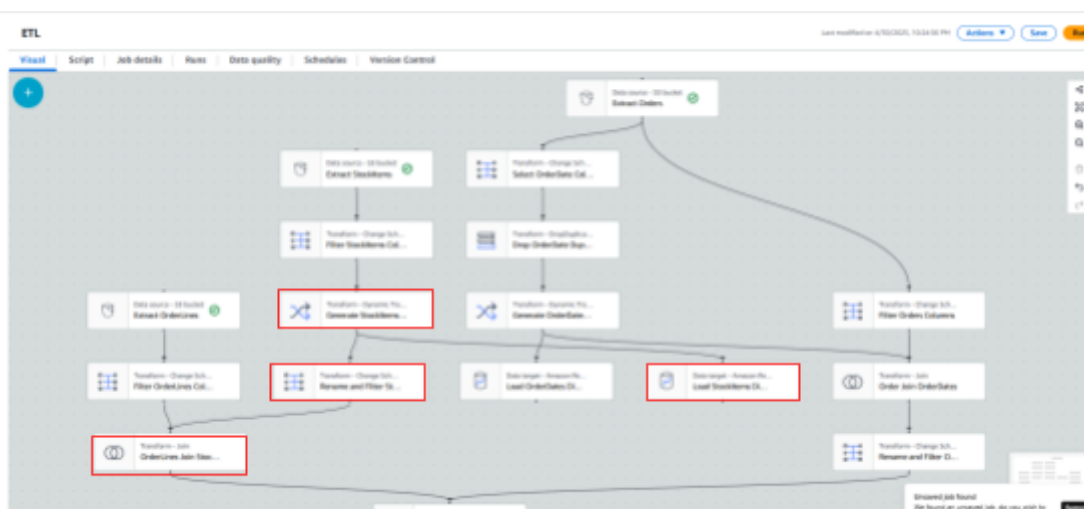
Ahora que hemos recapitulado en lo que hace cada nodo del diagrama, podemos pasar a describir de manera general el proceso que se lleva a cabo mediante todo el ETL.

En primer lugar, se realiza la extracción de los datos contenidos en tres archivos csv. Posteriormente, se realizan operaciones de filtrado sobre estos datos con fines específicos para cada dataset. Los

OrderLines tienen un solo filtro o transformación, en la cual se escogen los parámetros que se consideran necesarios del dataset, dejando atrás las columnas que no se consideran importantes. Los StockItems también pasan por un solo filtro en el cual sucede algo muy similar a lo anterior. No se deshacen de ninguna fila, pero sí quitan las columnas que no consideran apropiadas, dejando así solo aquellas que se consideran relevantes. Por otro lado, Orders cuenta con un proceso más complejo, ya que se realizan dos filtros diferentes sobre los datos. El primero se encarga de seleccionar todas las fechas asociadas a una orden de compra, y el segundo se encarga de descartar todas las columnas que no sean necesarias.

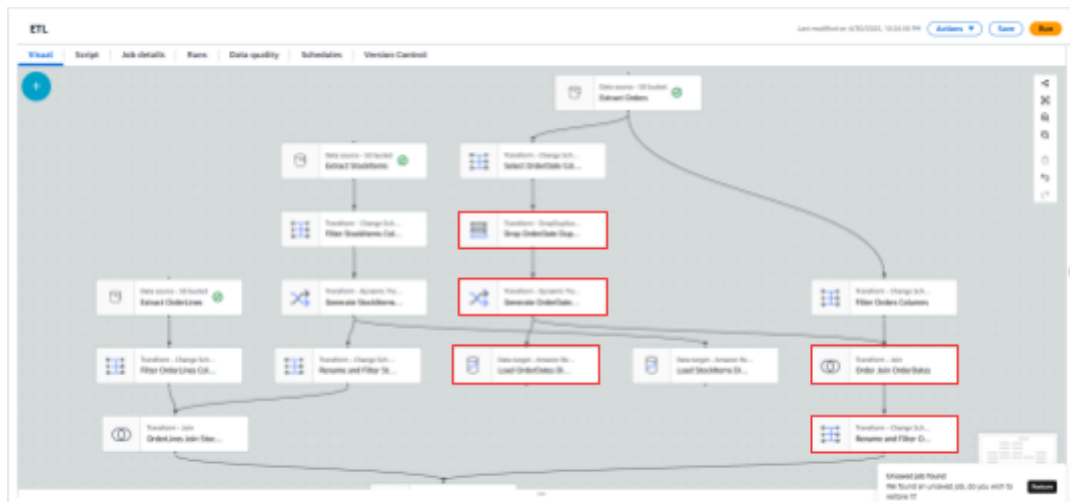


Una vez realizados estos filtros iniciales sobre los datos originales, observamos varias cosas. Primero, OrderLines, ahora filtrado, se pasa directamente a al proceso de unión con la tabla de StockItems, pero, antes de que StockItems se una con OrderLines, se le aplica una transformación UUID para generar la llave primaria que se será necesaria para identificar los productos. Luego, se le aplica otro filtro para renombrar los nombres de las columnas y que estos sean más apropiados en el momento de juntarse con la tabla de OrderLines. Es aquí donde ocurre el join entre estas dos tablas. De igual forma, la tabla modificada de StockItems se sube a redshift después de haber creado la llave primaria.

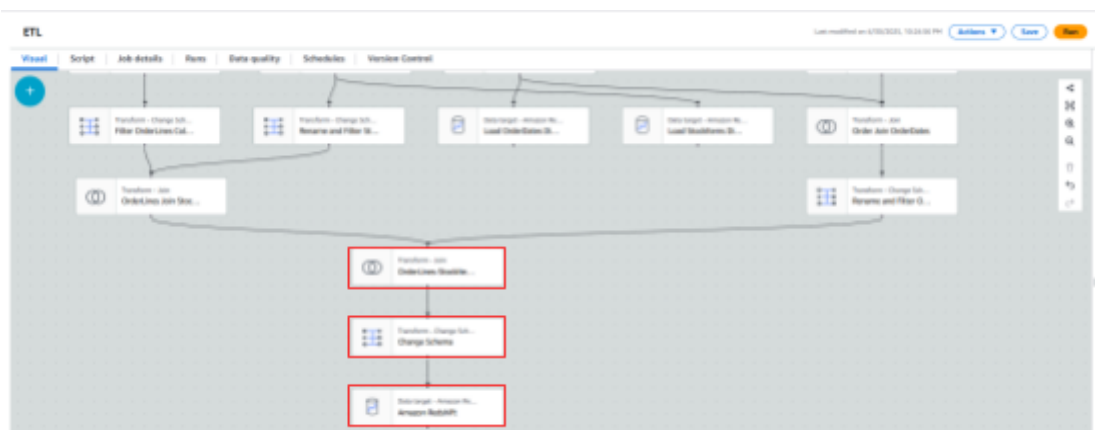


Ahora, el proceso de Extract Orders, que como mencionamos antes se realiza en dos filtros, pasa a su siguiente etapa. Primero, el filtro mediante el cual se obtenían solamente las fechas de las órdenes de

compra, ahora pasa por una transformación mediante la cual se eliminan las fechas duplicadas, dejando solamente fechas únicas. Con estas fechas únicas, se genera la llave primaria de la tabla que contendrá el registro de las fechas en las que se realizaron órdenes de compra. Una vez generada esta tabla con las llaves, se sube la tabla con las llaves primarias de las fechas (La tabla única) y también se genera una conexión con un nodo join al cual también apunta el segundo filtro utilizado para las órdenes, el cual dijimos que se utiliza para descartar las columnas que se consideran innecesarias. Una vez hecho el join, se filtra la combinación anterior para cambiar los nombres de algunas columnas y sacar otras que no son de interés.

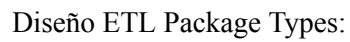


Finalmente, las tablas construidas y filtradas en los pasos anteriores mediante los joins, se juntan en una tabla compuesta que engloba todos los cambios propuestos. Construyendo así una tabla conjunta que describe la unión entre OrderLines-StockItems y OrderDates. De esta nueva tabla también se procede a hacer ciertas modificaciones sobre el esquema, quitando columnas que se consideran innecesarias, o se entiende que pueden llegar a generar inconvenientes. Por ejemplo, como en esta unión ya se encuentran todas las PKs, no tiene sentido tener ids, que era algo que antes si era necesario. Por eso, todas las columnas que describen id, son eliminadas. Finalmente, con la tabla lista, se hace la construcción de la tabla dentro de redshift, terminando el proceso y flujo del ETL.

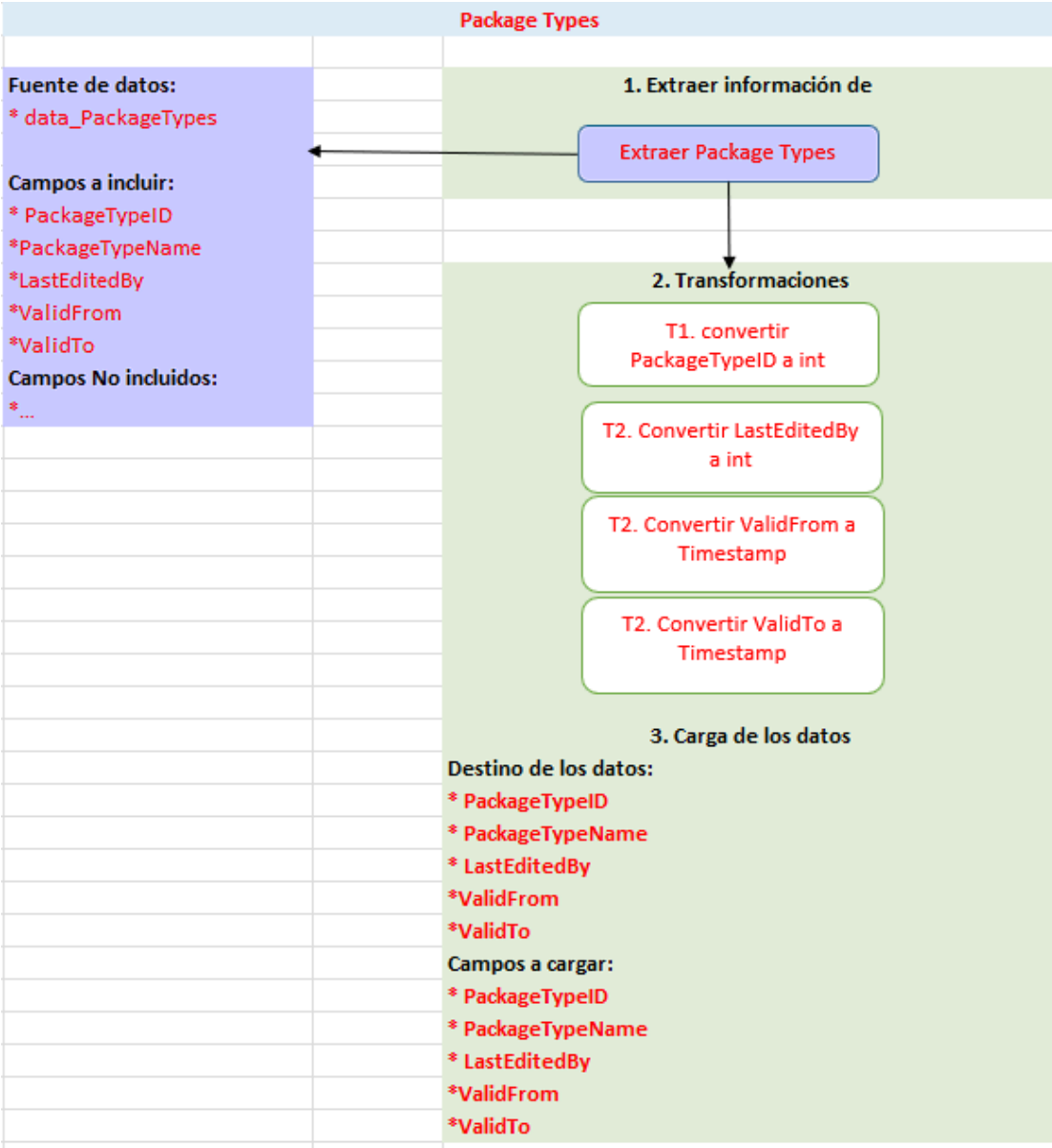


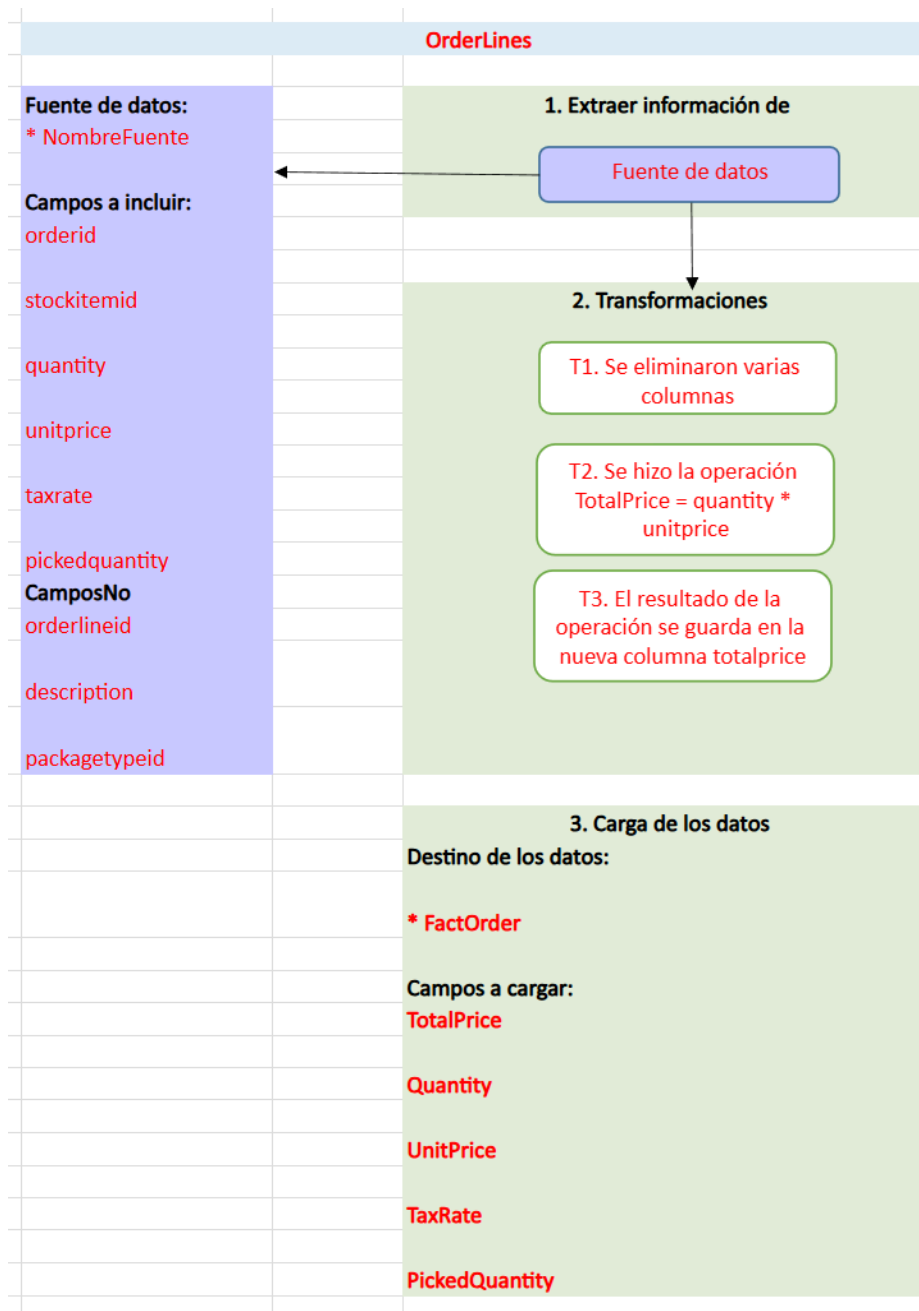
2.2 Extienda el ETL para incluir una nueva dimensión denominada **PackageTypes** utilizando el archivo CSV con el mismo nombre (que debió descargar previamente), más una nueva medida denominada TotalPrice la cual debe ser calculada multiplicando las columnas Quantity y UnitPrice del archivo OrderLines. Adjunte el diagrama de bloques final generado en Glue, así como el archivo

Dimensión PackageTypes:



	flask	type	no	exp	no-backup	no-backup-type	no-backup-size	validity	validity
✓	✓	packer-puppet	Integer	no	1	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	packer-puppet	Integer	no	2	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
		packer-puppet	Integer	no	3	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
		packer-puppet	Integer	no	4	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
✓	✓	last-snapshot	String (utf-8)	no	1	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	2	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	3	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	4	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
✓	✓	last-snapshot	String (utf-8)	no	5	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	6	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	7	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	last-snapshot	String (utf-8)	no	8	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
✓	✓	validity	String (utf-8)	no	9	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	10	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	11	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	12	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
✓	✓	validity	String (utf-8)	no	13	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	14	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	15	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59
	✓	validity	String (utf-8)	no	16	Black	2015-01-01-00:00:00	9999-12-31 23:59:59	9999-12-31 23:59:59





2.3 Revise las tablas creadas por el proceso ETL. Recuerde utilizar el usuario y contraseña que definió en la creación del clúster de Redshift como se muestra en la siguiente imagen.

R./

Etapa 3. Modelo Multidimensional y Análisis

3.1 Reconstruya el modelo multidimensional que se genera en Redshift tras la correcta ejecución del ETL con un ejercicio de ingeniería inversa.

Respuesta en imagen de repositorio

3.2 Diseña y documenta el resultado de al menos dos consultas SQL que involucren las medidas implementadas filtrando por algunas de las diferentes dimensiones disponibles.

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor on the left contains the following SQL query:

```
1 SELECT * FROM "dev"."public"."dim_packagetypes";
```

The results pane on the right displays the output of the query, which is a table with 14 rows and 5 columns. The columns are: packageid, packagetype, lasteditedby, validfrom, and validto. The data is as follows:

packageid	packagetype	lasteditedby	validfrom	validto
1	Bag	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
2	Block	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
3	Bottle	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
4	Box	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
5	Can	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
6	Carton	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
7	Each	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
8	Kg	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
9	Packlist	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
10	Pair	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
11	Pallet	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
12	Tray	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
13	Tub	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999
14	Tube	1	2013-01-01 00:00:00	9999-12-31 23:59:59.999

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor on the left contains the following SQL query:

```
1 SELECT * FROM "dev"."public"."fact_orders";
```

The results pane on the right displays the output of the query, which is a table with 100 rows and 10 columns. The columns are: orderid, orderdate, size, stockitemid, stockitemname, quantity, taxrate, unitofmeasure, and totalprice. The data is as follows:

orderid	orderdate	size	stockitemid	stockitemname	quantity	taxrate	unitofmeasure	totalprice
3	ab388521-c7af-41a5-bd34...	XS	6403c47f-8190-4442-92bd...	DXA joke mug - mind if I...	6	15	13	78
3	ab8a4813-9e0b-4b38-b5a...	XS	e5e43b04-6774-4471-82d...	The Cu red shirt XML tag...	36	15	18	648
3	6697037b-a022-4c59-aa0...	XL	665cabd5-a674-4695-978...	Developer joke mug - old...	8	15	13	104
3	33e56a73-d807-44b9-b0...	XL	26d70d29-a684-4d85-a2a...	Furry gorilla with big eyes...	2	15	32	64
3	d4b07516-a473-4882-aea...	1.5m	724b036e-6938-4977-8bb...	Bubblewrap dispenser (Bl...	1	15	240	240
3	18e14b0f-3288-4274-633...	XS	e9e43b04-6774-4471-82d...	The Cu red shirt XML tag...	108	15	18	1944
3	23031185-421c-4a2b-8cd...	1.5m	0b3c2a68-4f5e-4954-b19...	DXA joke mug - two types...	8	15	13	104
3	67012b48-b89c-4759-a76...	1.5m	367ec02a-1cc4-4b77-92b...	Office cube penelope (Bl...	100	15	18.5	1850
3	dae54bac-9ab3-4ba1-890...	1.5m	724b036e-6938-4977-8bb...	Bubblewrap dispenser (Bl...	1	15	240	240
3	60b02b2-aa33-4ea0-845...	1.5m	724b036e-6938-4977-8bb...	Bubblewrap dispenser (Bl...	7	15	240	1880
3	029c3850-7477-4b2a-a3a...	1.5m	665cabd5-a674-4695-978...	Developer joke mug - old...	9	15	13	117
3	208ac2a8-d9e8-4c04-b35...	1.5m	cd5582b6-6052-4685-911...	Developer joke mug - (hp...	10	15	13	130
3	858b67ab-89e8-4a67-93...	1.5m	367ec02a-1cc4-4b77-92b...	Office cube penelope (Bl...	40	15	18.5	740
3	8fa9e133-7b85-4a35-80c...	1.5m	bcd3a68b-b718-4b0b-a91...	The Cu red shirt XML tag...	72	15	18	1296
3	7db0124-43ca-4ba6-b19a...	1.5m	ec12ae62-2579-4c39-a85...	Furry gorilla with big eyes...	1	15	32	32

3.3 Finalmente, responda las siguientes preguntas:

¿Qué ventajas y desventajas observa al momento de implementar un ETL utilizando este tipo de herramientas respecto a desarrollarlo utilizando Python, Pandas y demás herramientas vistas durante la primera parte del curso?

Implementar un ETL con este tipo de herramientas resulta más sencillo y rápido que implementarlo con Python y otras herramientas de ese estilo. Esto se debe a que es una herramienta de muy alto nivel, de forma que el desarrollador no tiene que implementar tantas

cosas como lo tendría que hacer si se hiciera desde python. Sin embargo, esto también implica que las herramientas de AWS son menos flexibles que implementarlas desde Python. Las herramientas de AWS también permiten una estandarización de los procesos y comunicación fácil entre las herramientas de extracción, transformación y carga. Además, las herramientas de AWS le dan la posibilidad al desarrollador de enfocarse en la parte teórica y conceptual del modelado gracias a la interfaz visual, mientras que una implementación en Python también implicaría tiempo dedicado a el desarrollo de herramientas, estandarización, comunicación e integración con las bases de datos.

A partir del resto de información contenida en los archivos CSV proporcionados, ¿qué otros análisis considera que se pueden soportar y cómo se traducirían a dimensiones y medidas sobre el modelo actual?

Se podrían hacer más análisis sobre los tipos de productos dados, como filtrar por precio, encontrar la ganancia recomendada (costo del producto - precio recomendado), filtrar por tags. Para encontrar la ganancia recomendada, se haría una nueva medida para la dimensión stockitem, la cual podría llamarse recommendedprofit. Además, se le tendría que añadir la metrica de recommendedretailprice, la cual no se pone en los datos cargados pero está en la fuente de datos. Para los demás análisis no se tendrían que hacer cambios al modelo actual.

¿Qué errores se le presentaron en el desarrollo del laboratorio y qué solución plantearon? Haga énfasis en los que fueron más difíciles de solucionar.

- Elaboración de la dimensión Package Type del punto 2.2. En este punto nos dimos cuenta que agregar una dimensión no es un proceso tan fácil si no se tenía en cuenta desde el inicio. Para agregar la dimensión de Package Type se nos presentaron dificultades a la hora de conectarla ya que las reglas del ETL especificaban que varios nodos finales solo pueden tener máximo 2 nodos padres. Para resolver este problema se hizo una corrección sobre las tablas a las que entraban los datos, y se filtraron de forma que entraran con el tipo de dato correcto.